

UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA



División de Ingeniería Eléctrica
Departamento de Ingeniería en Computación
Computación Gráfica e Interacción Humano-Computadora

**EXAMEN EXTRAORDINARIO DE
LABORATORIO DE COMPUTACIÓN
GRÁFICA E INTERACCIÓN
HUMANO-COMPUTADORA**

Profesor: ING. JOSE RAMON PEREZ ATHIE

Alumno: Sánchez López Osvaldo

318211396

Semestre 2026-2

Fecha de entrega: 09 de junio de 2026

CALIFICACIÓN: _____

Índice

1. Configuración del Proyecto y Solución de Problemas	2
1.1. Descarga y Reemplazo de Recursos Externos	2
1.2. Configuración del Entorno en Visual Studio	2
1.3. Resolución de Errores y Dependencias Dinámicas (DLLs)	3
2. Corrección de materiales y UVs del escenario	4
2.1. Herramienta Seleccionada	4
2.2. Procedimiento Realizado	4
2.3. Resultados Obtenidos	4
3. Modificación de la Iluminación	5
3.1. Procedimiento Realizado	5
4. Modificación de la Animación de Personaje	6
4.1. Procedimiento Realizado	6
5. Integración y animación del modelo de zorro	6
5.1. Procedimiento Realizado	6
6. Integración de Modelos Adicionales	9
6.1. Procedimiento Realizado	9
6.2. Resultados Obtenidos	9
6.3. Procedimiento Realizado	10
6.4. Resultados Obtenidos	11
6.5. Animación Interactiva por Máquina de Estados (Modelo de Perro)	11
7. Referencias	12

1. Configuración del Proyecto y Solución de Problemas

En esta sección se detalla el proceso de preparación del entorno de desarrollo para el proyecto base provisto por el profesor sinodal, así como las correcciones adicionales realizadas para garantizar la correcta compilación y ejecución del programa.

1.1. Descarga y Reemplazo de Recursos Externos

Debido a las restricciones y la configuración de Git LFS (Large File Storage), la descarga del repositorio en formato `.zip` no incluye la totalidad de los archivos binarios y librerías necesarias. Por lo tanto, se procedió a realizar el reemplazo manual de los siguientes directorios utilizando los enlaces de respaldo proporcionados:

- **Librerías Externas:** Se descargó la carpeta completa de dependencias y se reemplazó en la ruta:
`ExtraOrdinario/External Libraries`
- **Modelos 3D:** Se descargó la carpeta con los assets tridimensionales y se colocó en la ruta:
`ExtraOrdinario/ExtraOrdinario/ModelsExtra`

1.2. Configuración del Entorno en Visual Studio

Para vincular correctamente las librerías al proyecto en el IDE Microsoft Visual Studio, se verificó y aseguró la siguiente configuración en las propiedades del proyecto, manteniendo estrictamente la plataforma de solución en **x86**:

1. **C/C++ → General → Additional Include Directories (Directorios de inclusión adicionales):**

```
$(SolutionDir)External Libraries\GLFW\Include
$(SolutionDir)External Libraries\GLEW\Include
$(SolutionDir)External Libraries\assimp\Include
$(SolutionDir)External Libraries\glm
$(SolutionDir)External Libraries\S0IL2\Include
```

2. **Linker → General → Additional Library Directories (Directorios de bibliotecas adicionales):**

```
$(SolutionDir)External Libraries\GLFW\lib-vc2015
$(SolutionDir)External Libraries\GLEW\lib\Release\Win32
$(SolutionDir)External Libraries\S0IL2\lib
$(SolutionDir)External Libraries\assimp\lib
```

3. **Linker → Input → Additional Dependencies (Dependencias adicionales):**

```
glfw3.lib
opengl32.lib
glew32.lib
soil2-debug.lib
assimp-vc140-mt.lib
```

1.3. Resolución de Errores y Dependencias Dinámicas (DLLs)

Durante la primera ejecución se presentó un error crítico señalando un conflicto con el archivo `glew32.dll`. Al inspeccionar el directorio del proyecto, se diagnosticó la ausencia y/o corrupción de ciertas librerías de enlace dinámico (.dll) esenciales para el *runtime*.

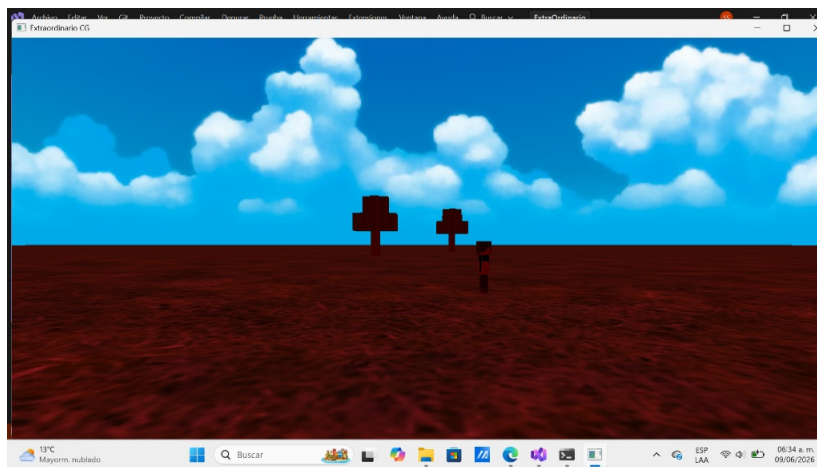
Para solucionar este inconveniente y lograr la ejecución exitosa del proyecto, se aplicaron las siguientes acciones correctivas:

1. **Corrección de GLEW (`glew32.dll`):** Al revisar la carpeta de librerías externas, el archivo `glew32.dll` no se encontraba en dicha ubicación. Para solucionar esto, se optó por utilizar el archivo binario proporcionado por el profesor de laboratorio del curso ordinario.

Nota: Alternativamente, en caso de no contar con dicho recurso, el archivo se puede obtener buscando “GLEW binaries” en el navegador para ingresar a la página oficial del proyecto (generalmente alojada en SourceForge), donde se puede descargar la versión estable más común (como la 2.1.0 o 2.2.0).

2. **Reubicación de Assimp (`assimp-vc140-mt.dll`):** Se identificó que el archivo `assimp-vc140-mt.dll` se encontraba correctamente en la carpeta origen `External Libraries/assimp`, pero estaba ausente en el directorio de salida de la aplicación (`Documentos\ExtraOrdinario\ExtraOrdinario`). Se realizó la copia manual del archivo a dicha locación.

Una vez colocadas ambas librerías dinámicas en el directorio correspondiente junto al binario ejecutable, el proyecto compiló y ejecutó de manera correcta.



2. Corrección de materiales y UVs del escenario

2.1. Herramienta Seleccionada

Se decidió utilizar el software de modelado 3D Blender debido a su soporte nativo para la manipulación de archivos Wavefront (.obj), su editor de coordenadas UV y su sistema de gestión de materiales independientes.

2.2. Procedimiento Realizado

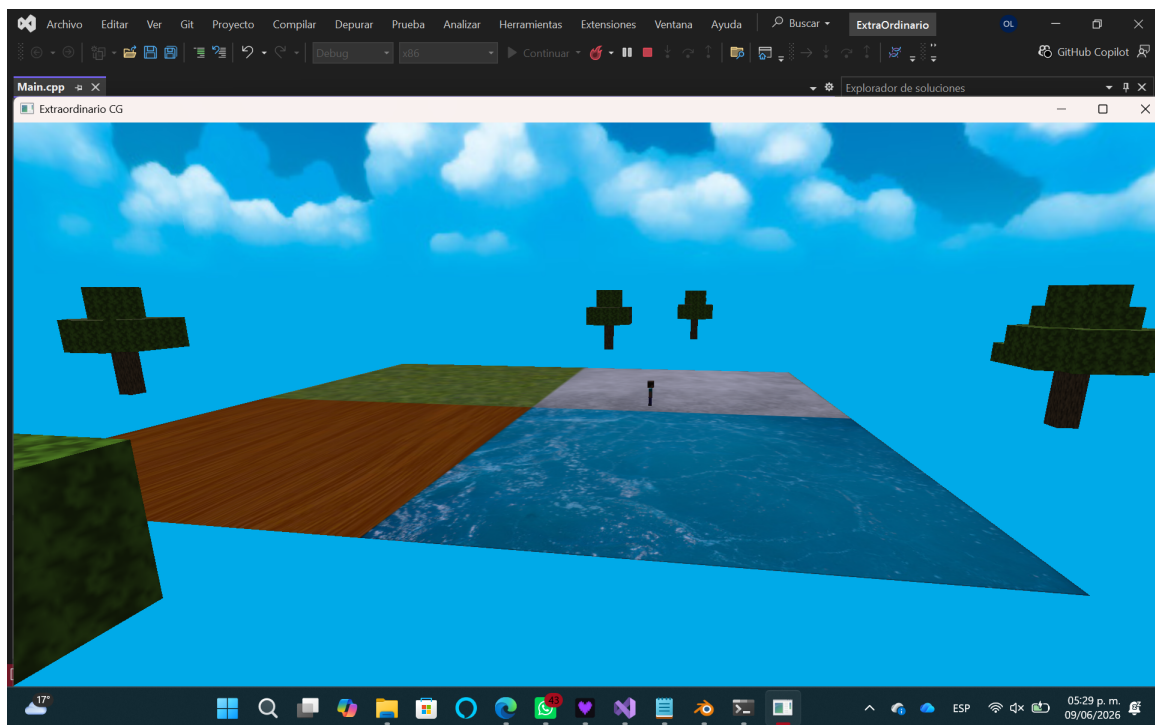
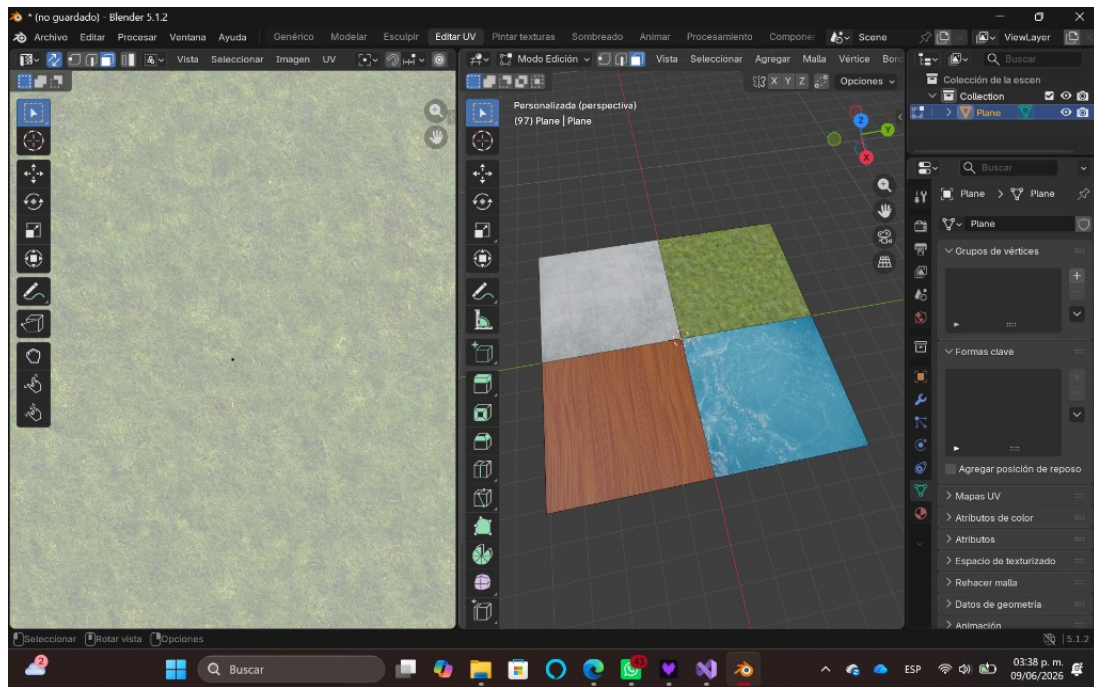
El desarrollo de la actividad se ejecutó mediante los siguientes pasos secuenciales:

1. **Limpieza de la Geometría:** Se importó el archivo original `Floor.obj` en Blender. Al identificar que el plano base estaba dividido por una arista diagonal (geometría triangulada), se aplicó la función de *disolver bordes* para unificar la superficie en una sola cara cuadrada limpia.
2. **Segmentación en Cuadrantes:** Con la cara limpia seleccionada, se empleó la herramienta de subdivisión con un factor de corte de 1. Esto dividió el plano original de forma simétrica en cuatro caras cuadradas de proporciones idénticas.
3. **Asignación de Materiales:** Se crearon cuatro materiales independientes en el panel de propiedades, nombrados de manera: **Pasto**, **Piedra**, **Madera** y **Agua**. A cada material se le vinculó su respectiva imagen en formato JPG mediante un nodo de textura de imagen, y se presionó el botón de asignar tras seleccionar la cara del cuadrante correspondiente.
4. **Ajuste del Mapeo UV:** Se ingresó al espacio de trabajo de edición UV para corregir la escala de visualización. Se seleccionaron las caras y se aplicó la opción de reiniciar (*Reset*) el mapa UV, lo que obligó a las coordenadas de cada cuadrante a expandirse en el rango absoluto de 0.0 a 1.0. Esto evitó distorsiones y aseguró que cada textura cubriera su cara completa a la escala correcta.

2.3. Resultados Obtenidos

Se obtuvo un nuevo modelo geométrico `FloorUpdate.obj` y su archivo complementario `FloorUpdate.mtl`. El piso quedó estructurado en cuatro cuadrantes simétricos, cada uno con un material independiente y una textura asignada correctamente. Al colocar estos archivos junto con las imágenes JPG en la ruta del proyecto, la clase `Model` en C++ realiza la lectura de las sub-mallas y las texturas de forma automática, permitiendo un renderizado del suelo que coincide con la referencia solicitada. Se probó el modelo modificando la siguiente línea del proyecto:

```
//Model Aldea  
Model Aldea((char*)"ModelsExtra/FloorUpdate.obj");
```

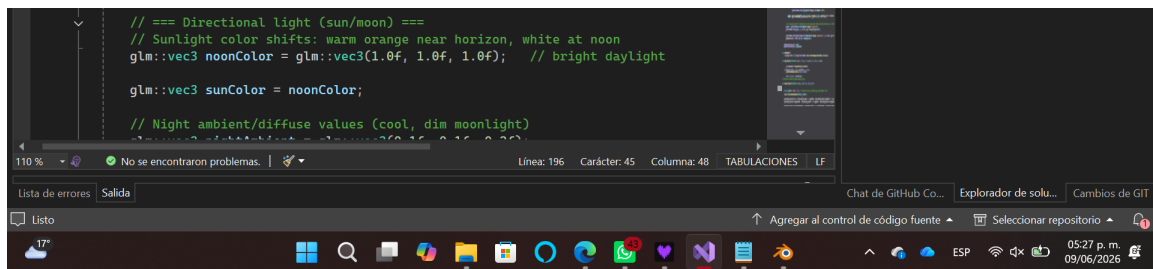


3. Modificación de la Iluminación

3.1. Procedimiento Realizado

Se analizó el archivo `Main.cpp` para localizar la configuración correspondiente a la luz direccional del escenario. Se identificó la variable que define el color base de dicha luz, la cual emplea un vector de tres dimensiones para el modelo de color RGB. Para lograr la emisión de luz blanca, se asignó el valor máximo a los tres canales (Rojo, Verde y Azul),

modificando la línea de código para establecer el vector como `glm::vec3(1.0f, 1.0f, 1.0f)`.



```
// == Directional light (sun/moon) ==
// Sunlight color shifts: warm orange near horizon, white at noon
glm::vec3 noonColor = glm::vec3(1.0f, 1.0f, 1.0f); // bright daylight

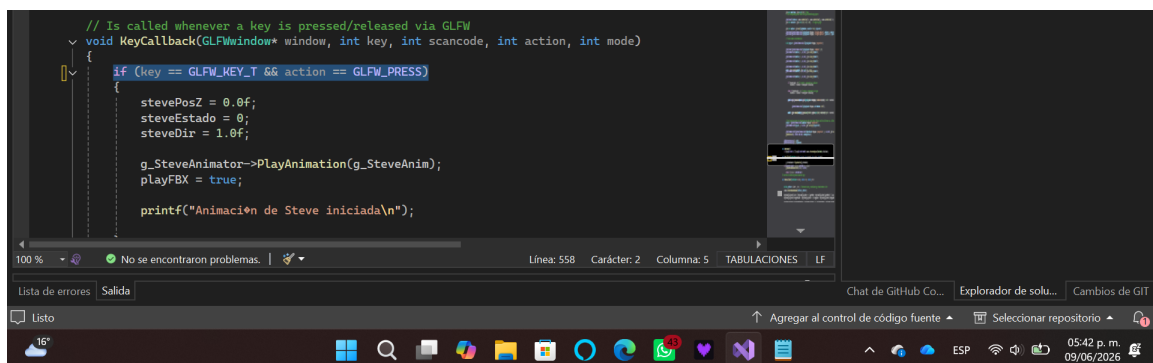
glm::vec3 sunColor = noonColor;

// Night ambient/diffuse values (cool, dim moonlight)
```

4. Modificación de la Animación de Personaje

4.1. Procedimiento Realizado

Se analizó el archivo `Main.cpp` para localizar la función `KeyCallback`, la cual gestiona las interrupciones del teclado. Dentro de las condicionales de esta función, se identificó el bloque de código responsable de reiniciar los parámetros de traslación del personaje “Steve”, activar la bandera de estado `playFBX` y disparar el método `PlayAnimation()`. Se reemplazó la constante original de activación `GLFW_KEY_B` por la nueva constante solicitada `GLFW_KEY_T`.



```
// Is called whenever a key is pressed/released via GLFW
void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode)
{
    if (key == GLFW_KEY_T && action == GLFW_PRESS)
    {
        stevePosZ = 0.0f;
        steveEstado = 0;
        steveDir = 1.0f;

        g_SteveAnimator->PlayAnimation(g_SteveAnim);
        playFBX = true;

        printf("Animación de Steve iniciada\n");
    }
}
```

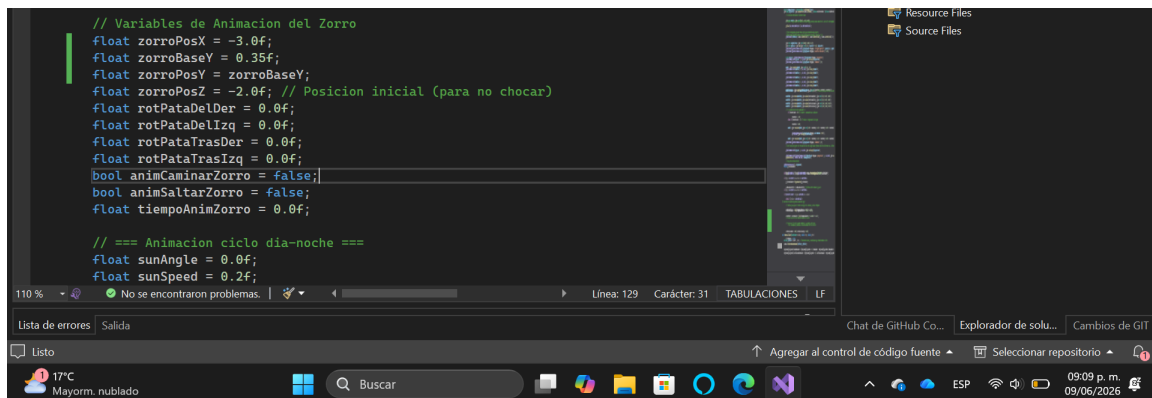
5. Integración y animación del modelo de zorro

5.1. Procedimiento Realizado

Para cumplir con los requerimientos de la actividad, se modificó el código fuente `Main.cpp` para integrar las distintas partes del modelo del zorro y controlar sus transformaciones espaciales. El procedimiento se dividió en los siguientes pasos:

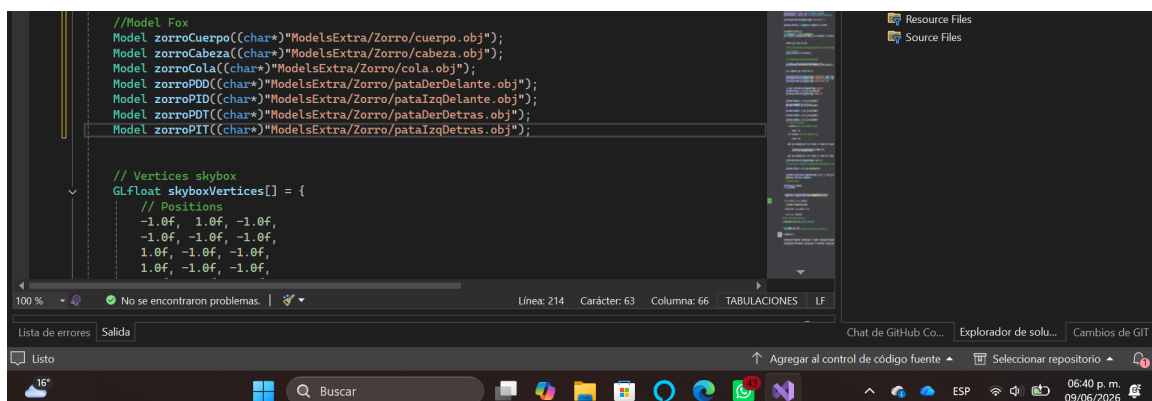
1. **Integración y ajuste de geometría en la escena:** Se inicializaron y renderizaron los múltiples archivos `.obj` que componen al zorro (cabeza, cuerpo, cola y patas) vinculándolos a una matriz de transformación base (`glm::mat4`). Para corregir el problema de intersección geométrica donde el modelo aparecía enterrado en la textura del suelo, se definió una variable de compensación en el eje Y (`zorroBaseY = 0.35f`), ajustando el origen del modelo al nivel de la superficie.

2. **Animación procedural por código:** Se programaron dos animaciones continuas dentro de la función `AnimationKeys()`:
 - *Caminar:* Se implementó un desplazamiento lineal y constante sobre el eje Z. La rotación alternada de las patas delanteras y traseras se calculó utilizando la función trigonométrica `sin()` multiplicada por el factor de tiempo (`deltaTime`), simulando la marcha del modelo.
 - *Saltar:* Se programó un desplazamiento vertical en el eje Y evaluando una parábola mediante la función `sin()`, sumado a un avance frontal en Z. Durante esta animación, las patas rotan a un ángulo fijo para simular el impulso, y la posición en Y retorna de manera estricta al valor de `zorroBaseY` al finalizar el ciclo para evitar desfases.
3. **Prevención de colisiones:** Para evitar que el modelo avanzara indefinidamente y chocara con los objetos del entorno (como los árboles) o saliera del área de renderizado, se implementó una validación de límites espaciales. Cuando la posición del zorro supera el límite frontal (`zorroPosZ > 6.0f`), sus coordenadas se reinician al punto de partida, estableciendo un ciclo (loop) infinito y seguro.
4. **Asignación de controles:** Dentro de la función `KeyCallback()`, se condicionó la lectura del teclado para que las teclas **Z** y **X** accionen los estados booleanos `animCaminarZorro` y `animSaltarZorro` respectivamente. Al presionar cualquiera de las teclas, se reinicia la variable de tiempo local para garantizar que las animaciones comiencen desde el cuadro cero.



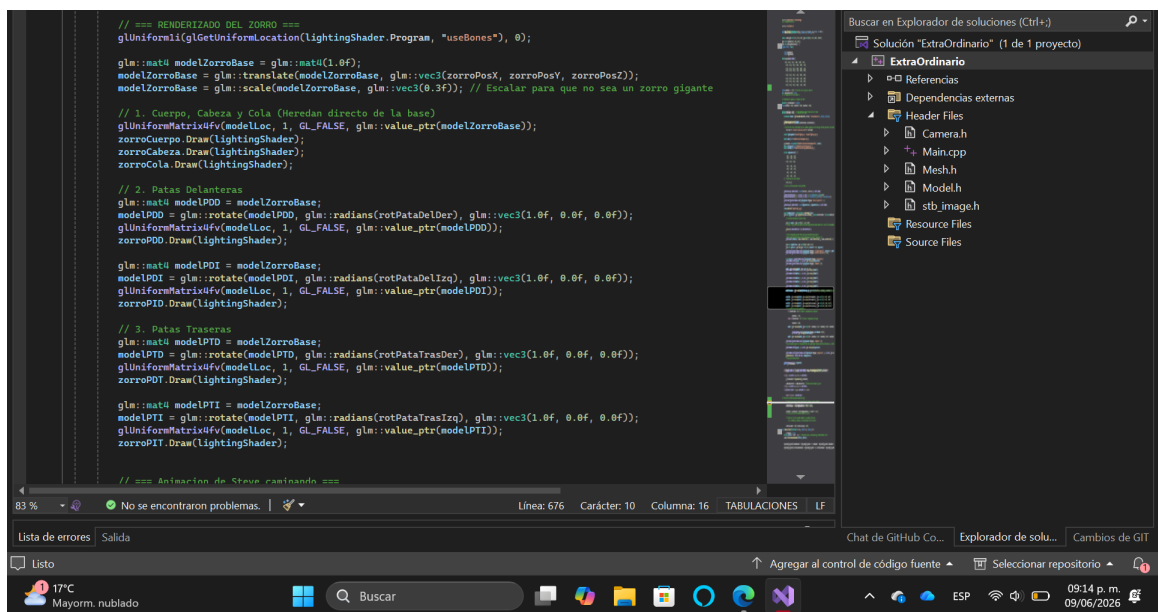
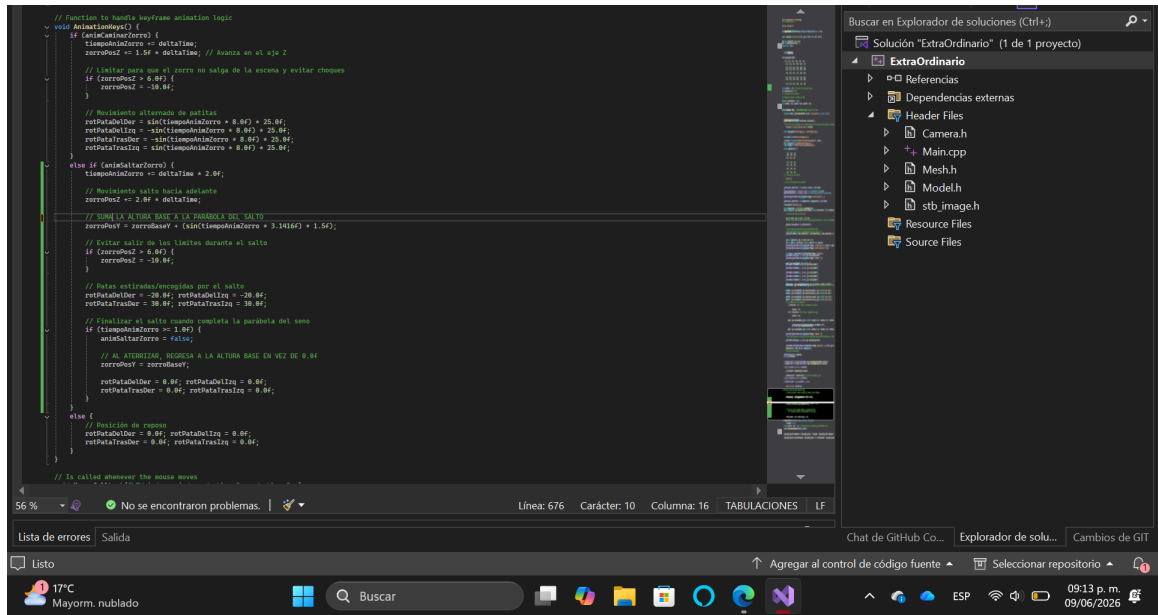
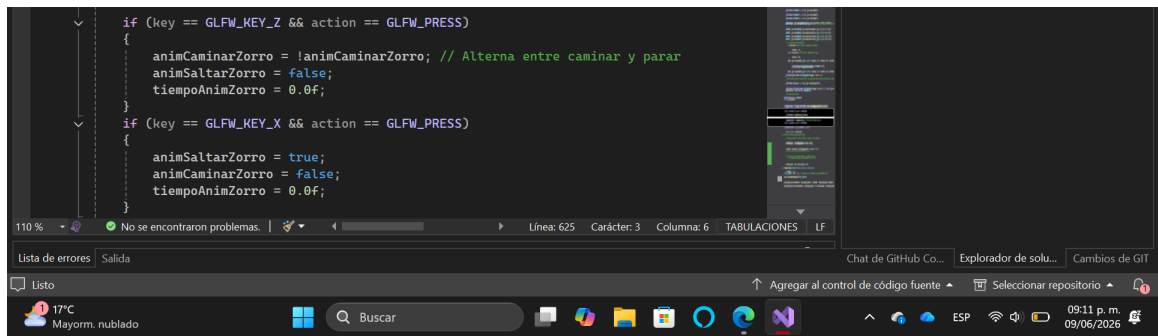
```
// Variables de Animacion del Zorro
float zorroPosX = -3.0f;
float zorroBaseY = 0.35f;
float zorroPosY = zorroBaseY;
float zorroPosZ = -2.0f; // Posicion inicial (para no chocar)
float rotPataDelDer = 0.0f;
float rotPataDelIzq = 0.0f;
float rotPataTrasDer = 0.0f;
float rotPataTrasIzq = 0.0f;
bool animCaminarZorro = false;
bool animSaltarZorro = false;
float tiempoAnimZorro = 0.0f;

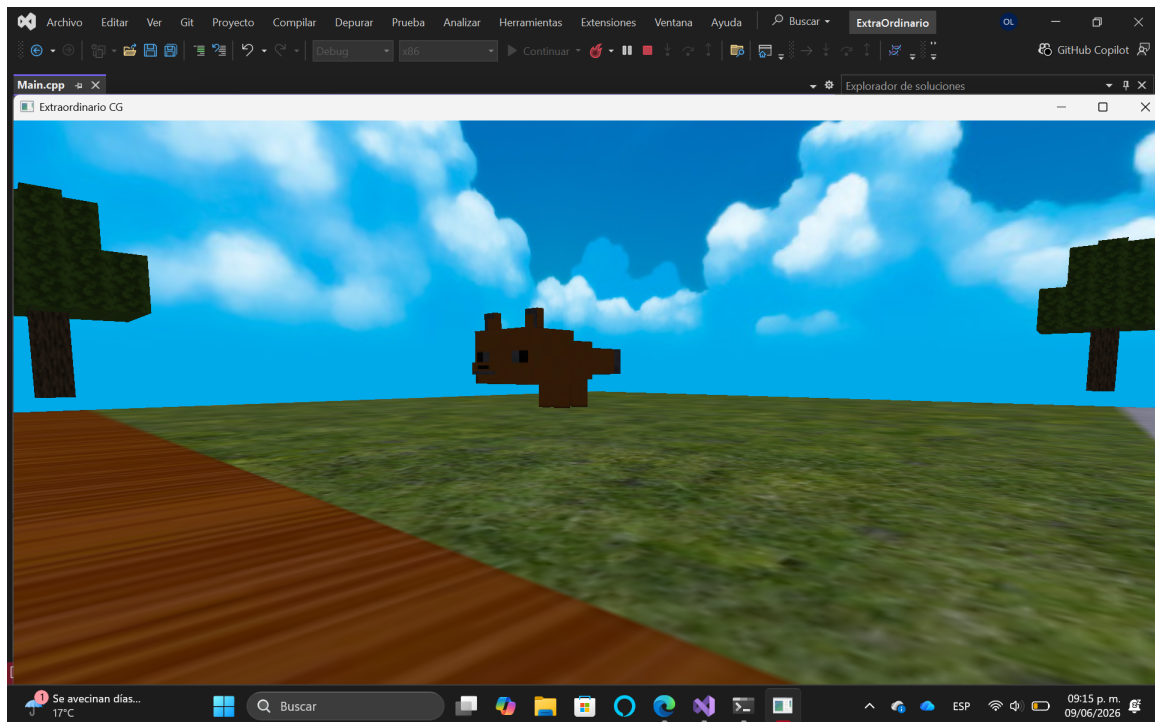
// === Animacion ciclo dia-noche ===
float sunAngle = 0.0f;
float sunSpeed = 0.2f;
```



```
//Model Fox
Model zorroCuerpo((char*)"ModelsExtra/Zorro/cuerpo.obj");
Model zorroCabeza((char*)"ModelsExtra/Zorro/cabeza.obj");
Model zorroCola((char*)"ModelsExtra/Zorro/cola.obj");
Model zorroPDD((char*)"ModelsExtra/Zorro/pataDerDelante.obj");
Model zorroPID((char*)"ModelsExtra/Zorro/pataIzqDelante.obj");
Model zorroPDT((char*)"ModelsExtra/Zorro/pataDerDetras.obj");
Model zorroPIT((char*)"ModelsExtra/Zorro/pataIzqDetras.obj");

// Vertices skybox
GLfloat skyboxVertices[] = {
    // Positions
    -1.0f, 1.0f, -1.0f,
    -1.0f, -1.0f, -1.0f,
    1.0f, -1.0f, -1.0f,
    1.0f, 1.0f, -1.0f,
    -1.0f, 1.0f, 1.0f,
    -1.0f, -1.0f, 1.0f,
    1.0f, -1.0f, 1.0f,
    1.0f, 1.0f, 1.0f
};
```





6. Integración de Modelos Adicionales

Modelos estáticos

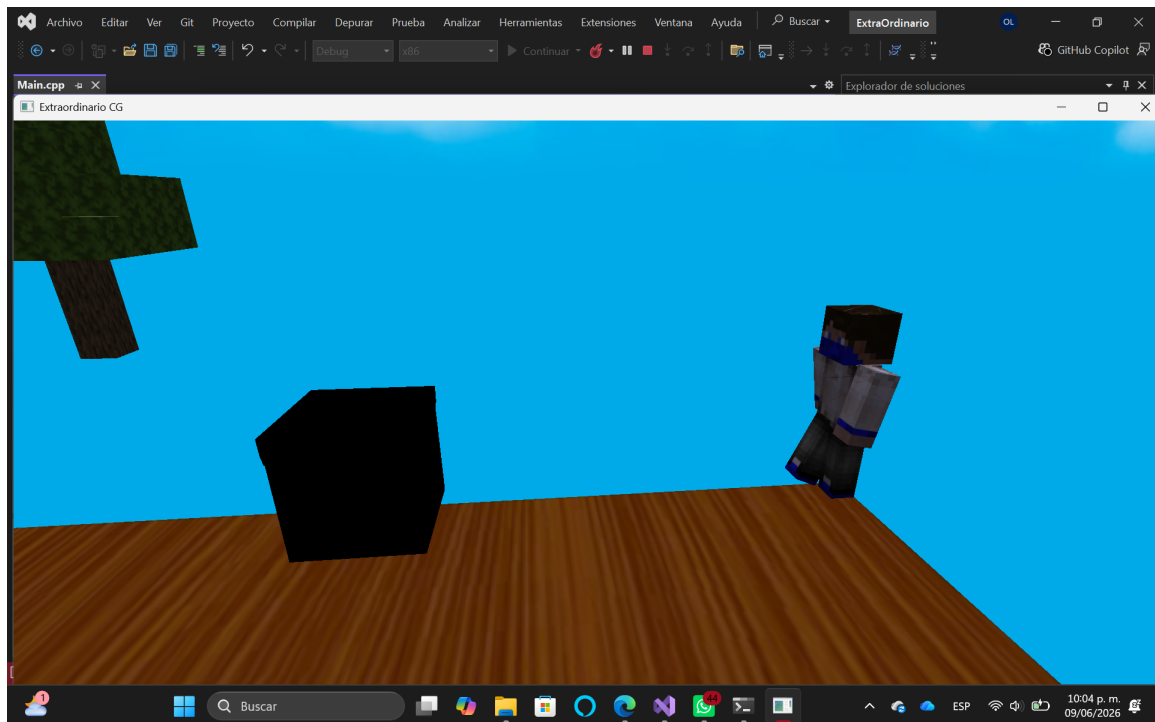
Para complementar el escenario gráfico con la temática solicitada, se procedió a la descarga de modelos 3D de repositorios web externos. Debido a restricciones operativas (no me dejaban descargar modelos) y de tiempo de desarrollo, el alcance de esta fase se limitó a la integración de dos modelos estáticos en formato `.obj`, denominados `model_low` y `Grass_block`.

6.1. Procedimiento Realizado

Se agregaron los archivos `model_low.obj` y `Grass_block.obj` a la carpeta de recursos del proyecto. En el código de la aplicación, se realizó la carga de ambos archivos mediante la clase `Model`. Posteriormente, dentro del ciclo principal de renderizado, se aplicaron matrices individuales de traslación y escalado para ubicar los objetos en el espacio, y se mandó a llamar al método `Draw` de cada modelo para dibujarlos en la pantalla.

6.2. Resultados Obtenidos

Se obtuvo la visualización exitosa de ambos modelos estáticos dentro de la escena de OpenGL, quedando integrados correctamente junto al resto de los elementos del escenario gráfico.



Modelos con animación

Para complementar el escenario gráfico, se buscaron modelos 3D en repositorios web externos. Debido a la dificultad para encontrar modelos y animaciones decentes y compatibles relacionados con la temática de Minecraft, se optó por utilizar el modelo `ball.obj` proporcionado previamente en el material de las clases de laboratorio. Adicionalmente, se mantuvieron los modelos estáticos `model_low.obj` y `Grass_block.obj`.

6.3. Procedimiento Realizado

Se agregaron los archivos de los modelos a la carpeta de recursos del proyecto y se cargaron en el código mediante la clase `Model`. Posteriormente, se trabajó en la animación en bucle continuo de la bola:

1. **Variables globales:** Se declararon variables para controlar el ángulo de rotación (`anguloBola`), la distancia al centro (`radioBola`) y la rapidez del giro (`velocidadBola`).
// Variables Animacion Bola (animación en bucle continuo)
float `anguloBola` = 0.0f;
float `radioBola` = 2.0f; // Distancia a la que girará del zorro
float `velocidadBola` = 2.0f;
2. **Lógica del bucle:** Dentro de la función `AnimationKeys()`, se programó un incremento constante del ángulo usando el factor de tiempo (`deltaTime`). Para evitar desbordamientos de memoria a largo plazo, se implementó una condición que reinicia el ángulo a 0 cuando supera los 360 grados.

3. **Transformaciones espaciales:** En el ciclo de renderizado, se aplicaron operaciones matriciales en un orden específico para lograr el efecto de órbita. Primero, la bola se trasladó a la posición actual del zorro (sumando un valor en Y para ajustarla a la altura del torso); luego, se rotó sobre el eje Y utilizando la variable del ángulo; y finalmente, se trasladó hacia un lado en el eje X para definir el radio del giro.

6.4. Resultados Obtenidos

Se obtuvo la visualización exitosa de los modelos estáticos dentro de la escena de OpenGL. Además, se logró que el modelo de la bola gire de manera infinita alrededor del zorro, cumpliendo satisfactoriamente con el requisito de implementar una animación que se reproduzca continuamente en bucle.

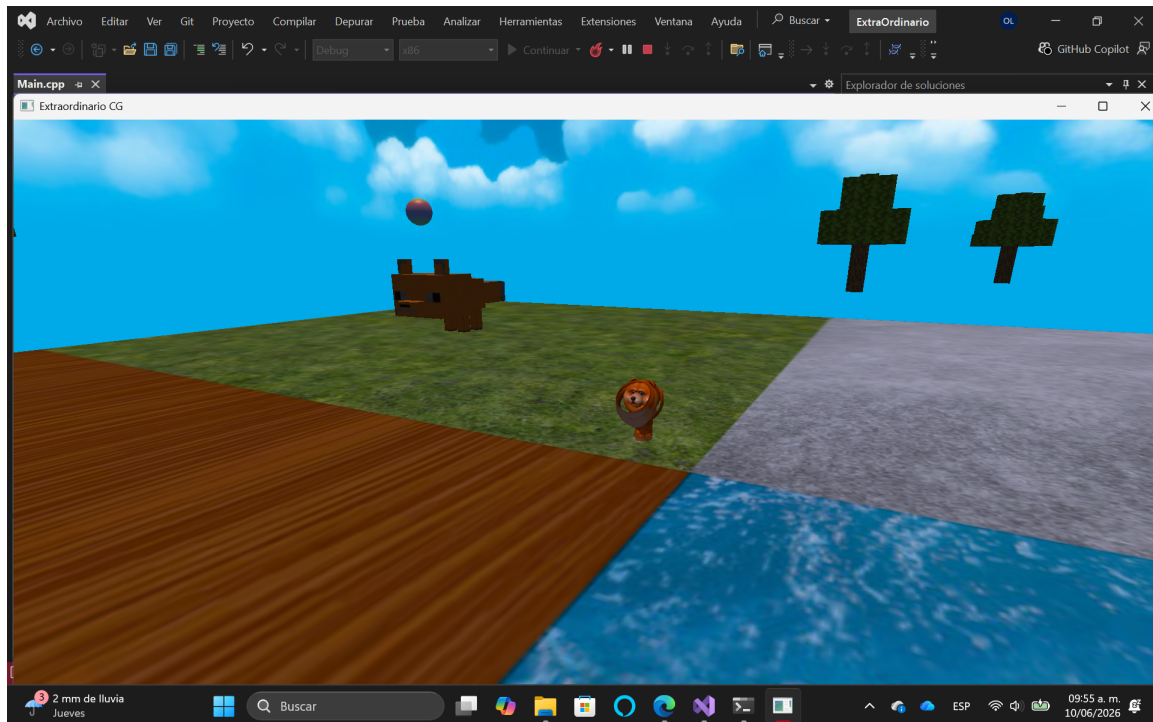
6.5. Animación Interactiva por Máquina de Estados (Modelo de Perro)

Para cumplir con el segundo requisito de animación con interacción del usuario, se integró en la escena el modelo de un perro rojo dividido en múltiples piezas independientes. La lógica de control y su comportamiento se estructuraron de la siguiente manera:

1. **Máquina de estados y control por teclado:** Se implementó una estructura lógica mediante un `enum` para gestionar dos estados básicos: **CAMINANDO** y **PARADO**. La transición entre estos estados se configuró dentro de la función `KeyCallback()` para responder a las interrupciones del teclado en tiempo real:
 - **Tecla C:** Cambia el estado del modelo a **CAMINANDO**, lo que reanuda el avance y las oscilaciones de las piezas.
 - **Tecla V:** Cambia el estado a **PARADO**, lo que detiene por completo el paso del tiempo local y congela las transformaciones del personaje.
2. **Trayectoria en bucle continuo:** Cuando el estado está activo, el perro se desplaza de forma lineal en línea recta sobre el eje Z. Para cumplir con el comportamiento en

bucle infinito, se programó una condicional de límite espacial; al momento en que el modelo supera la coordenada frontal de la escena (`perroPosZ > 6.0f`), su posición se restablece de golpe al fondo del mapa, repitiendo el ciclo de marcha de forma indefinida.

3. **Documentación de limitaciones en el ensamble jerárquico:** Después de demasiados intentos por ensamblar el perro no se pudo y quedó un tanto deforme, sin embargo, se dejó tal y como está ara asegurar la entrega y aún así cumpliendo con los requerimientos.



7. Referencias

CGTrader. (s.f.-a). *Man in Minecraft style 0111* [Modelo 3D]. Recuperado de <https://www.cgtrader.com/free-3d-models/character/fantasy-character/man-in-minecraft-style-0111-99a1c94c-d332-4279-a113-494643f4d28d>

CGTrader. (s.f.-b). *Minecraft grass block* [Modelo 3D]. Recuperado de <https://www.cgtrader.com/free-3d-models/plant/grass/minecraft-grass-block-eea105f3-c4e0-4eec-82d8-0ef85282f6bd>

GDT Solutions ES. (2022, 31 de enero). { *Algunas formas de SUBDIVIDIR un MESH en Blender* } [Video]. YouTube. https://www.youtube.com/watch?v=yUc8-Cp8y_c

Tecnologías Interactivas y Computación Gráfica. (2024a, 8 de octubre). *Animación Básica en OpenGL* [Video]. YouTube. <https://www.youtube.com/watch?v=Iuybf6bTzhc>

Tecnologías Interactivas y Computación Gráfica. (2024b, 16 de octubre). *Animación con Máquinas de Estados en OpenGL: Movimiento de Objetos 3D [Tutorial Completo]* [Video]. YouTube. <https://www.youtube.com/watch?v=B-mqD317HHM>

Tecnologías Interactivas y Computación Gráfica. (2024c, 22 de octubre). *Animación por Keyframes en OpenGL Movimiento y Rotación Suaves [Tutorial Completo]* [Video]. YouTube. <https://www.youtube.com/watch?v=511A6Lxa3GQ>

Tecnologías Interactivas y Computación Gráfica. (2024d, 7 de septiembre). *Carga de Modelos 3D y Cámara Sintética en OpenGL* [Video]. YouTube. <https://www.youtube.com/watch?v=iZu3z2E4qKs>

Tecnologías Interactivas y Computación Gráfica. (2024e, 14 de septiembre). *Carga de Texturas en OpenGL Introducción a las UV s* [Video]. YouTube. https://www.youtube.com/watch?v=21j0b3w_T8k&t=2s

Tecnologías Interactivas y Computación Gráfica. (2024f, 30 de agosto). *Modelado Jerárquico en OpenGL: Creación de un Brazo Robótico* [Video]. YouTube. <https://www.youtube.com/watch?v=pq5Ikb8TFQo>